

Module 1 — Automation with Selenium

Student Name	Shreya Ghosh
Course	B.Tech — Computer Science and Business Systems (CSBS) (4 th Year)
Enrollment No.	12023052018010
Programming Language	Python
Automation Tool	Selenium WebDriver
Academic Year	2026

Experiment 1 — The Multi-Locator Challenge

1. Problem Statement

Automate a login page by locating the username field using ID, the password field using Name, and the login button using XPath. Verify successful navigation after login.

2. Objective

To demonstrate the use of multiple Selenium locator strategies in one automation script and validate the resulting page URL.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Google Chrome and ChromeDriver
- SauceDemo practice application
- Locators: By.ID, By.NAME, By.XPATH
- send_keys(), click(), current_url, assert
- time.sleep() for visible execution during this assignment

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

try:

    driver.get("https://www.saucedemo.com/")

    driver.maximize_window()

    time.sleep(2)

    username = driver.find_element(By.ID, "user-name")
    username.send_keys("standard_user")

    print("Username entered using By.ID")
    time.sleep(2)
    password = driver.find_element(By.NAME, "password")
    password.send_keys("secret_sauce")

    print("Password entered using By.NAME")
    time.sleep(2)
    login_button = driver.find_element(
```

```

        By.XPATH, "//input[@type='submit' and @value='Login']"
    )

    print("Login button located using By.XPATH")
    time.sleep(2)

    login_button.click()

    print("Login button clicked")
    time.sleep(3)
    assert "/inventory.html" in driver.current_url

    print("Login successful!")
    print("Current URL:", driver.current_url)
    print("Assignment 1 completed successfully.")

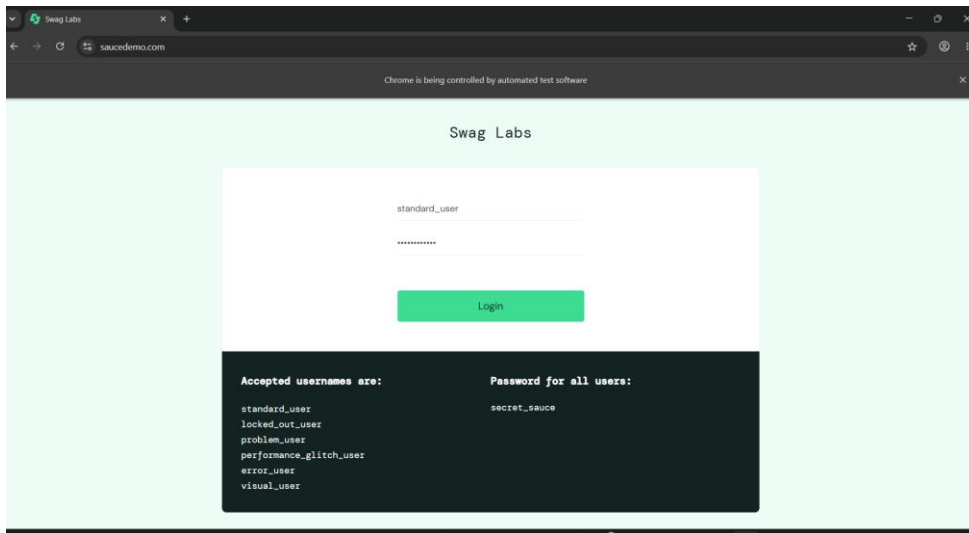
    time.sleep(3)

finally:
    driver.quit()

```

5. Output

The terminal displayed successful username and password entry, button interaction, and a successful login with the inventory URL.



6. Result

The login automation completed successfully and the expected inventory page was reached.

7. Observation

Different locator strategies can be combined within the same Selenium test according to the attributes available on the target elements.

8. Conclusion

The experiment demonstrated practical element identification and basic Selenium WebDriver interaction.

Experiment 2 — Synchronization & Explicit Waits

1. Problem Statement

Automate a page where content is loaded dynamically through AJAX. The script must wait for the dynamic content to become available without using `time.sleep()`.

2. Objective

To demonstrate explicit synchronization using `WebDriverWait` and expected conditions before extracting dynamically loaded text.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- WebDriverWait
- expected_conditions (EC)
- By.ID
- AJAX/dynamic content handling
- `element_to_be_clickable()`
- `text_to_be_present_in_element()`

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    driver.get(
        "https://testautomationpractice.blogspot.com/"
        "p/gui-elements-ajax-hidden.html"
    )

    driver.maximize_window()

    wait = WebDriverWait(driver, 10)

    load_button = wait.until(
        EC.element_to_be_clickable(
            (By.ID, "loadContent")
        )
    )

    print("Load AJAX Content button found.")

    load_button.click()
```

```
print("Load AJAX Content button clicked.")
```

```
wait.until(  
    EC.text_to_be_present_in_element(  
        (By.ID, "ajaxContent"),  
        "AJAX Content Loaded"  
    )  
)
```

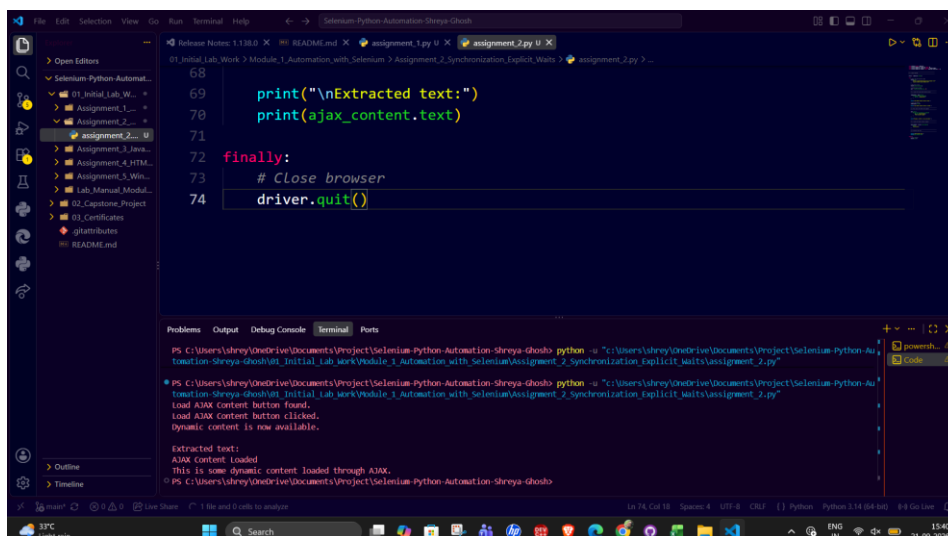
```
print("Dynamic content is now available.")
```

```
ajax_content = driver.find_element(  
    By.ID,  
    "ajaxContent"  
)  
print("\nExtracted text:")  
print ajax_content.text
```

```
finally:  
    driver.quit()
```

5. Output

The script located and clicked the AJAX button, waited for the expected text, and extracted the dynamically loaded content.



6. Result

The dynamic content was successfully synchronized and extracted without using `time.sleep()`.

7. Observation

Explicit waits allow the script to wait for a specific browser condition instead of relying on a fixed delay.

8. Conclusion

The experiment demonstrated reliable synchronization with dynamically changing web content.

Experiment 3 — Dynamic Dropdowns & Checkboxes

1. Problem Statement

Automate a flight booking form containing multiple checkboxes and a search-as-you-type country autocomplete field. Select specific checkboxes, verify their states, and choose a matching autocomplete result.

2. Objective

To demonstrate checkbox selection and state verification along with handling dynamically generated autocomplete suggestions.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- WebDriverWait
- expected_conditions
- By.ID
- By.CSS_SELECTOR
- is_selected()
- send_keys()
- Dynamic autocomplete suggestions

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    driver.get("https://rahulshettyacademy.com/dropdownsPractise/")
    driver.maximize_window()

    wait = WebDriverWait(driver, 10)

    family_checkbox = driver.find_element(
        By.ID,
        "ctl00_mainContent_chk_friendsandfamily"
    )

    family_checkbox.click()

    print(
        "Family and Friends selected:",
        family_checkbox.is_selected()
    )

    assert family_checkbox.is_selected()
```

```

student_checkbox = driver.find_element(
    By.ID,
    "ctl00_mainContent_chk_StudentDiscount"
)

student_checkbox.click()

print(
    "Student selected:",
    student_checkbox.is_selected()
)

assert student_checkbox.is_selected()
country_box = wait.until(
    EC.element_to_be_clickable(
        (By.ID, "autosuggest")
    )
)

country_box.send_keys("ind")

print("Typed 'ind' in Country box.")

suggestions = wait.until(
    EC.visibility_of_all_elements_located(
        (By.CSS_SELECTOR, "li.ui-menu-item a")
    )
)

print("Number of suggestions:", len(suggestions))

for suggestion in suggestions:

    print("Suggestion:", suggestion.text)

    if suggestion.text == "India":

        suggestion.click()

        print("India selected successfully.")

        break

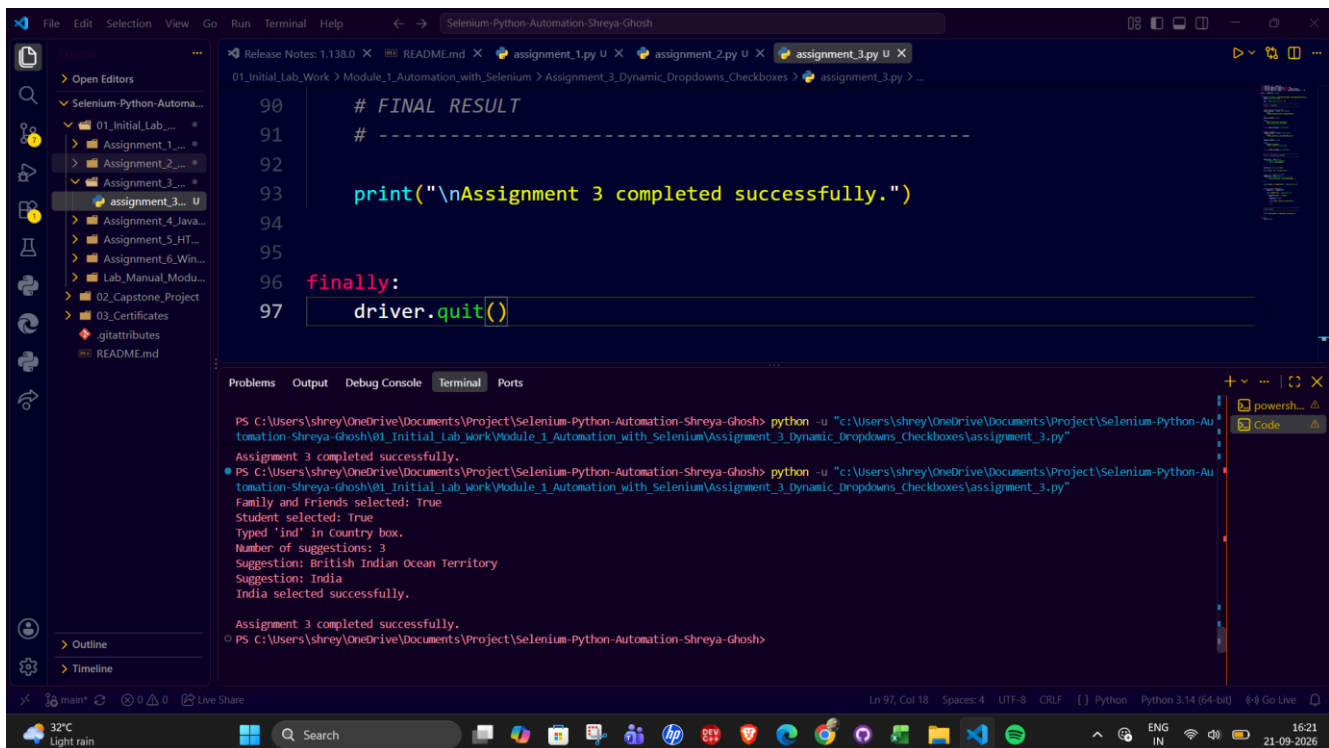
print("\nAssignment 3 completed successfully.")

finally:
    driver.quit()

```

5. Output

The selected checkbox states were verified as True, the country field accepted the input 'ind', and the India suggestion was selected.



6. Result

The required checkboxes and dynamic dropdown option were handled successfully.

7. Observation

Specific element IDs are more reliable than selecting all checkbox inputs when a page contains hidden or additional checkbox elements.

8. Conclusion

The experiment demonstrated handling of interactive controls and dynamically generated autocomplete results.

Experiment 4 — JavaScript Alerts and Confirms

1. Problem Statement

Trigger and handle a JavaScript alert, a confirmation box, and a prompt box on a practice page.

2. Objective

To demonstrate Selenium's alert handling methods for accepting alerts, dismissing confirms, and entering text into a prompt.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- WebDriverWait
- expected_conditions.alert_is_present()
- By.LINK_TEXT
- Alert text extraction

- `accept()`, `dismiss()`, `send_keys()`

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    driver.get("https://www.selenium.dev/selenium/web/alerts.html")
    driver.maximize_window()

    wait = WebDriverWait(driver, 10)
    alert_link = driver.find_elements(
        By.LINK_TEXT,
        "click me"
    )[0]

    alert_link.click()

    alert = wait.until(
        EC.alert_is_present()
    )

    print("Alert message:", alert.text)
    alert.accept()

    print("JavaScript Alert accepted successfully.")

    confirm_link = driver.find_element(
        By.LINK_TEXT,
        "test confirm"
    )

    confirm_link.click()

    confirm = wait.until(
        EC.alert_is_present()
    )

    print("Confirm message:", confirm.text)

    confirm.dismiss()

    print("Confirm box dismissed successfully.")
    prompt_link = driver.find_element(
        By.LINK_TEXT,
        "prompt happen"
    )
```

```

prompt_link.click()

prompt = wait.until(
    EC.alert_is_present()
)

print("Prompt message:", prompt.text)

prompt.send_keys("Shreya")

print("Text entered into prompt.")

prompt.accept()

print("Prompt submitted successfully.")

print("\nAssignment 4 completed successfully.")

finally:
    driver.quit()

```

5. Output

The terminal showed the alert message, confirm message, and prompt message. The alert was accepted, the confirm was dismissed, and text was entered and submitted in the prompt.

The screenshot shows a VS Code editor with a Python script named 'assignment_4.py' open. The script contains the following code:

```

91
92     print("\nAssignment 4 completed successfully.")
93
94
95 finally:
96     driver.quit()

```

The terminal output shows the execution of the script, which includes the following messages:

```

PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh> python -u "C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_1_Automation_with_Selenium\Assignment_4_JavaScript_Alerts_Confirms\assignment_4.py"

Assignment 3 completed successfully.
PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh> python -u "C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_1_Automation_with_Selenium\Assignment_4_JavaScript_Alerts_Confirms\assignment_4.py"
Alert message: cheese
JavaScript Alert accepted successfully.
Confirm message: Are you sure?
Confirm box dismissed successfully.
Prompt message: Enter something
Text entered into prompt.
Prompt submitted successfully.

Assignment 4 completed successfully.
PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh>

```

6. Result

All three JavaScript dialog types were handled successfully.

7. Observation

Selenium exposes the active JavaScript dialog through the alert interface and provides separate actions for accepting, dismissing, and entering prompt text.

8. Conclusion

The experiment demonstrated reliable handling of JavaScript browser dialogs.

Experiment 5 — The HTML Web Table Extractor

1. Problem Statement

Find a multi-column web table, iterate through its rows and cells, locate a specific book by name, and retrieve the corresponding value from the Price column.

2. Objective

To demonstrate systematic extraction of table headers, rows, columns, and a matching cell value from an HTML web table.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- By.XPATH
- By.TAG_NAME
- HTML table structure: table, tr, th, td
- List iteration
- String matching
- Column-index identification

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("https://testautomationpractice.blogspot.com/")
    driver.maximize_window()

    table = driver.find_element(
        By.XPATH,
        "//table[.//th[normalize-space()='BookName']]")
    print("Web table found successfully.")

    headers = table.find_elements(By.TAG_NAME, "th")
```

```

header_names = []

for header in headers:
    header_names.append(header.text.strip())

print("\nTable Headers:")
print(header_names)
price_index = header_names.index("Price")

print("Price column index:", price_index)
rows = table.find_elements(By.TAG_NAME, "tr")

print("\nTotal rows including header:", len(rows))
target_book = "Learn Java"
book_found = False

for row in rows[1:]:

    cells = row.find_elements(By.TAG_NAME, "td")

    if len(cells) == 0:
        continue
    row_data = []

    for cell in cells:
        row_data.append(cell.text.strip())

    print("Row:", row_data)

    # Match the BookName
    if row_data[0] == target_book:

        book_found = True

        price = row_data[price_index]

        print("\n-----")
        print("Target Book:", target_book)
        print("Price:", price)
        print("-----")

        break
assert book_found, f"{target_book} was not found."

print("\nAssignment 5 completed successfully.")

finally:
    driver.quit()

```

5. Output

The table headers were extracted as BookName, Author, Subject, and Price. The row for Learn Java was found and its Price value was extracted as 500.

```
89 assert book_found, f"{target_book} was not found."
90
91 print("\nAssignment 5 completed successfully.")
92
93 finally:
94     driver.quit()
```

Problems Output Debug Console Terminal Ports

PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh> python -u "c:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_1_Automation_with_Selenium\Assignment_4_JavaScript_Alerts_confirms\assignment_4.py"

tomation-Shreya-Ghosh\01_Initial_Lab_Work\Module_1_Automation_with_Selenium\Assignment_5_HTML_Table_Extractor\assignment_5.py"

Web table found successfully.

Table Headers:
['BookName', 'Author', 'Subject', 'Price']
Price column index: 3

Total rows including header: 7
Row: ['Learn Selenium', 'Amit', 'Selenium', '300']
Row: ['Learn Java', 'Mukesh', 'Java', '500']

Target Book: Learn Java
Price: 500

Assignment 5 completed successfully.

○ PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh>

6. Result

The target row was located successfully and the corresponding Price value of 500 was retrieved.

7. Observation

Determining the column index from the header makes the extraction more structured than assuming a fixed column position.

8. Conclusion

The experiment demonstrated row-wise and column-wise extraction from an HTML web table.

Experiment 6 — Windows, Tabs and Iframes

1. Problem Statement

Automate a page containing an embedded iframe and a link that opens a new browser window. Switch into the iframe, read its content, return to the main document, switch to the new window, obtain its title, close it, and return to the original window.

2. Objective

To demonstrate Selenium context switching between an iframe and multiple browser windows.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- WebDriverWait

- expected_conditions
- By.NAME and By.LINK_TEXT
- switch_to.frame()
- switch_to.default_content()
- current_window_handle
- window_handles
- switch_to.window()
- close()

4. Source Code / Implementation

The following implementation was executed using Selenium WebDriver with Python.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    driver.get(
        "https://www.selenium.dev/selenium/web/"
        "window_switching_tests/page_with_frame.html"
    )

    driver.maximize_window()

    wait = WebDriverWait(driver, 10)

    print("Practice page opened successfully.")

    main_window = driver.current_window_handle

    print("Main window handle stored.")

    iframe = wait.until(
        EC.presence_of_element_located(
            (By.NAME, "myframe")
        )
    )

    driver.switch_to.frame(iframe)

    print("Switched to iframe successfully.")

    iframe_body = driver.find_element(
        By.TAG_NAME,
        "body"
    )

    print("Text inside iframe:")
```

```
print(iframe_body.text)

iframe_body.click()

print("Interacted with the element inside the iframe successfully.")

driver.switch_to.default_content()

print("Switched back to main page.")

open_window = driver.find_element(
    By.LINK_TEXT,
    "Open new window"
)

open_window.click()

print("Open new window link clicked.")

wait.until(
    EC.number_of_windows_to_be(2)
)

print(
    "Number of browser windows:",
    len(driver.window_handles)
)

for window in driver.window_handles:
    if window != main_window:
        driver.switch_to.window(window)
        break

print("Switched to new window successfully.")

new_window_title = driver.title

print("New window title:", new_window_title)

driver.close()

print("New window closed successfully.")

driver.switch_to.window(main_window)

print("Switched back to main window.")

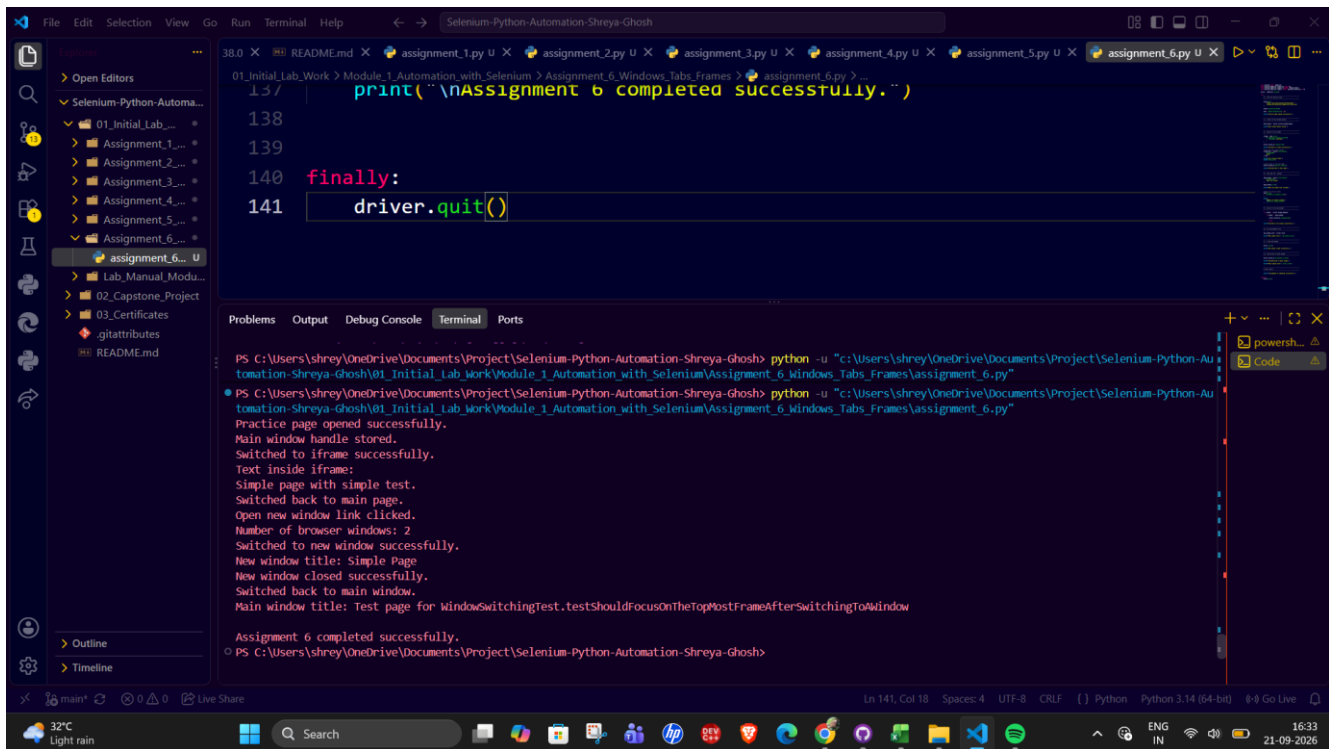
print("Main window title:", driver.title)

print("\nAssignment 6 completed successfully.")

finally:
    driver.quit()
```

5. Output

The script successfully switched into the iframe and extracted 'Simple page with simple test.' It then opened a second window, switched to it, retrieved the title 'Simple Page', closed it, and returned to the main window.



The screenshot shows a VS Code editor window with a Python script named `assignment_6.py` in the editor. The script contains the following code:

```
137 print("\nAssignment 6 completed successfully.")
138
139
140 finally:
141     driver.quit()
```

The terminal output shows the execution of the script, which includes the following steps:

- PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh> python -u "c:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_1_Automation_with_Selenium\Assignment_6_Windows_Tabs_Frames\assignment_6.py"
- Practice page opened successfully.
- Main window handle stored.
- Switched to iframe successfully.
- Text inside iframe:
- Simple page with simple test.
- Switched back to main page.
- Open new window link clicked.
- Number of browser windows: 2
- Switched to new window successfully.
- New window title: Simple Page
- New window closed successfully.
- Switched back to main window.
- Main window title: Test page for WindowSwitchingTest.testShouldFocusOnTheTopMostFrameAfterSwitchingToAWindow
- Assignment 6 completed successfully.

The terminal output also shows the command prompt prompt `PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh>` at the end of the execution.

6. Result

The iframe and browser-window operations completed successfully.

7. Observation

Frames and windows are separate browsing contexts; Selenium requires explicit context switching before working in a different frame or window.

8. Conclusion

The experiment demonstrated browser context management using Selenium WebDriver.